

El proyecto tiene ya resuelta toda la base sobre la que se construirá el resto del sistema. Está cerrada la parte normativa —certificado de piloto A1/A3 y número de operador AESA— y el acceso a un espacio de vuelo seguro, gracias a la federación en el club Alas de Galapagar, cuya licencia de socio incluye el seguro obligatorio. He realizado además un primer vuelo de referencia, cuyo registro GPS me sirve para planificar misiones en simulación.

En el plano técnico, la infraestructura extremo a extremo está operativa. La adquisición de datos en el nodo edge se ha organizado por dominios independientes (ambiental, sistema, vuelo y detección), cada uno como un proceso propio con su buffer local en SQLite y publicando bajo una estructura jerárquica de topics `sa/{dron_id}/{dominio}`, con almacenamiento intermedio local para no perder datos ante cortes de red. El sensor ambiental BME680 sobre la Raspberry Pi 5 alimenta ya el dominio ambiental, cuyas lecturas llegan por MQTT al servidor en la nube (AWS EC2 con Mosquitto e InfluxDB). El dominio y los servidores web están en marcha, y tanto las páginas como los servicios dinámicos (API REST e InfluxDB) se sirven ya por HTTPS con certificado propio, accediéndose a estos últimos a través de un reverse proxy en nginx que permite mantener cerrados sus puertos al exterior.

En cuanto al vuelo, he llevado el control autónomo de la simulación a la Raspberry Pi: la Pi comanda el dron simulado (Mission Planner + ArduPilot SITL) por MAVLink a través de la red. Sobre esa base he montado el control por MQTT —un publicador en el EC2 y un receptor en la Pi que traduce las órdenes a MAVLink— y una API REST con una página de botones para teleoperar el dron (armar, despegar, aterrizar, RTL, misión) desde el navegador, ya validada contra el simulador. Este panel se ha ampliado además con un selector que permite elegir a qué dron (dron-01 / dron-02) se dirige cada comando, y con nuevos botones de gestión remota del nodo edge —como apagar la Raspberry Pi o ajustar en caliente parámetros de funcionamiento (cada cuánto envía datos el dominio de sistema o el intervalo *anti-spam* de las alertas)—. Tanto la API como el receptor se ejecutan como servicios systemd, de modo que arrancan solos y se reinician ante cualquier interrupción.

En el apartado de visión artificial, he desarrollado el pipeline de detección de personas con YOLO y lo he validado sobre vídeo real, ejecutándolo en mi propio equipo (todavía sin el acelerador Hailo y sin desplegar en la Raspberry Pi). Cuando el sistema detecta a una persona, genera una alerta que se publica por MQTT con el mismo esquema *store-and-forward* que el resto de dominios, incluyendo la caja de la detección, la posición del dron y una foto del fotograma con las coordenadas y la marca temporal superpuestas. En paralelo, he empezado a construir el dataset con Roboflow y ya he capturado las primeras imágenes para el entrenamiento.

El repositorio del servidor se ha reorganizado por carpetas para separar las páginas web, el panel de control y la API, y se trabaja ahora sobre una rama develop que mantiene la rama principal con la versión estable. El fallo de la tarjeta microSD que dejó temporalmente fuera de servicio a la Raspberry Pi está resuelto, se ha sustituido y repetido la instalación de la imagen, además de realizar nuevamente el script del receptor, ya que no estaba subido dicho código.

Próximos pasos: ampliar el dataset de detección con muchas más imágenes aéreas para entrenar el modelo definitivo, y montar la VPN que dé conectividad segura entre el nodo edge y la nube. En paralelo, ensamblar el dron definitivo (pendiente de comprar el material) e incorporar autenticación a la API y cifrado MQTT antes de conectar la cadena de mando a la aeronave real.